Topic 12: Integrated Swift Programming

Task 1: Library Lending Model

```
protocol Borrowable {
    var id: Int { get }
    var title: String { get }
    var isAvailable: Bool { get set }
}

struct Book: Borrowable {
    let id: Int
    let title: String
    let author: String
    var isAvailable = true
}

struct Magazine: Borrowable {
    let id: Int
    let title: String
    let issue: Int
    var isAvailable = true
}

enum LibraryError: Error {
    case itemNotFound(id: Int)
    case alreadyBorrowed(title: String)
    case limitReached(max: Int)
}

final class Member {
    let name: String
    private(set) var borrowedIDs: [Int] = []
    static let limit = 2

    init(name: String) {
        self.name = name
    }

    func recordBorrow(_ id: Int) throws {
        if borrowedIDs.count >= Member.limit {
            throw LibraryError.limitReached(max: Member.limit)
        }
        borrowedIDs.append(id)
    }

    func recordReturn(_ id: Int) {
        borrowedIDs.removeAll { $0 == id }
    }
}

struct Catalog<Item: Borrowable> {
    private var items: [Int: Item] = [:]
```

```
    mutating func add(_ item: Item) {
        items[item.id] = item
    }

    mutating func checkout(id: Int, to member: Member) throws {
        guard var item = items[id] else {
            throw LibraryError.itemNotFound(id: id)
        }

        guard item.isAvailable else {
            throw LibraryError.alreadyBorrowed(title: item.title)
        }

        try member.recordBorrow(id)
        item.isAvailable = false
        items[id] = item
    }
}

var catalog = Catalog<Book>()
catalog.add(Book(id: 1, title: "Swift Basics", author: "John"))
catalog.add(Book(id: 2, title: "Java Guide", author: "David"))
catalog.add(Book(id: 3, title: "Python Intro", author: "Sam"))

let member = Member(name: "Gayu")

do {
    try catalog.checkout(id: 1, to: member)
    print("Book 1 borrowed successfully")

    try catalog.checkout(id: 1, to: member)
} catch LibraryError.alreadyBorrowed(let title) {
    print("\(title) is already borrowed")
} catch {
    print("Error:", error)
}

do {
    try catalog.checkout(id: 2, to: member)
    print("Book 2 borrowed successfully")

    try catalog.checkout(id: 3, to: member)
} catch LibraryError.limitReached(let max) {
    print("Borrowing limit reached: \(max) items")
} catch {
    print("Error:", error)
}

print("Borrowed IDs:", member.borrowedIDs)
```

Output:



Task 2: Generic Repository

```
protocol Entity {
    var id: Int { get }
}

enum RepositoryError: Error {
    case duplicateID(Int)
    case notFound(Int)
}

struct Repository<T: Entity> {
    private var storage: [Int: T] = [:]

    mutating func add(_ item: T) throws {
        if storage[item.id] != nil {
            throw RepositoryError.duplicateID(item.id)
        }
        storage[item.id] = item
    }

    func find(_ id: Int) -> T? {
        return storage[id]
    }

    mutating func remove(_ id: Int) throws {
        guard storage[id] != nil else {
            throw RepositoryError.notFound(id)
```

```
        }
        storage.removeValue(forKey: id)
    }

    func all(where condition: (T) -> Bool) -> [T] {
        return storage.values
            .filter(condition)
            .sorted { $0.id < $1.id }
    }
}

struct Student: Entity {
    let id: Int
    let name: String
    let marks: Int
}

struct Course: Entity {
    let id: Int
    let title: String
}

var students = Repository<Student>()

do {
    try students.add(Student(id: 2, name: "Ravi", marks: 75))
    try students.add(Student(id: 1, name: "Asha", marks: 90))
    try students.add(Student(id: 3, name: "Kiran", marks: 60))
    try students.add(Student(id: 1, name: "Priya", marks: 80))
} catch RepositoryError.duplicateID(let id) {
    print("Duplicate student ID:", id)
} catch {
    print("Error:", error)
}

if let student = students.find(1) {
    print("Found:", student.name)
}

let toppers = students.all { $0.marks >= 75 }

for student in toppers {
    print(student.id, student.name, student.marks)
```

```
}

do {
    try students.remove(5)
} catch RepositoryError.notFound(let id) {
    print("Student not found:", id)
} catch {
    print("Error:", error)
}

var courses = Repository<Course>()

do {
    try courses.add(Course(id: 101, title: "Swift"))
    try courses.add(Course(id: 102, title: "Java"))
} catch {
    print("Error:", error)
}

for course in courses.all(where: { _ in true }) {
    print(course.id, course.title)
}
```

Output:

```
Duplicate student ID: 1
Found: Asha
1 Asha 90
2 Ravi 75
Student not found: 5
101 Swift
102 Java
```

Task 3: Reports Through Protocol Extensions

```
protocol Entity {
    var id: Int { get }
}

protocol Describable {
    var summary: String { get }
}
```

```
enum RepositoryError: Error {
    case duplicateID(Int)
    case notFound(Int)
}

struct Student: Entity, Describable {
    let id: Int
    let name: String
    let marks: Int

    var summary: String {
        return "Student \(id): \(name), Marks: \(marks)"
    }
}

struct Course: Entity, Describable {
    let id: Int
    let title: String

    var summary: String {
        return "Course \(id): \(title)"
    }
}

struct Repository<T: Entity> {
    private var storage: [Int: T] = [:]

    mutating func add(_ item: T) throws {
        if storage[item.id] != nil {
            throw RepositoryError.duplicateID(item.id)
        }
        storage[item.id] = item
    }

    func find(_ id: Int) -> T? {
        return storage[id]
    }

    mutating func remove(_ id: Int) throws {
        guard storage[id] != nil else {
            throw RepositoryError.notFound(id)
        }
```

```
        storage.removeValue(forKey: id)
    }

    func all(where condition: (T) -> Bool) -> [T] {
        return storage.values
            .filter(condition)
            .sorted { $0.id < $1.id }
    }
}

extension Repository where T: Describable {
    func printReport() {
        for item in all(where: { _ in true }) {
            print(item.summary)
        }
    }
}

var students = Repository<Student>()
var courses = Repository<Course>()

do {
    try students.add(Student(id: 1, name: "Asha", marks: 95))
    try students.add(Student(id: 2, name: "Ravi", marks: 80))

    try courses.add(Course(id: 101, title: "Swift"))
    try courses.add(Course(id: 102, title: "Java"))
} catch {
    print("Error:", error)
}

print("Student Report")
students.printReport()

print("Course Report")
courses.printReport()
```

output:

Output

```
Student Report
Student 1: Asha, Marks: 95
Student 2: Ravi, Marks: 80
Course Report
Course 101: Swift
Course 102: Java
```